

Exercice 1 :**Deux implémentations des graphes**

On donne dans le fichier `graphes.mli` l'interface d'une structure mutable de graphe orienté¹. On souhaite pouvoir :

- pour $n \in \mathbb{N}$, créer un graphe (S, A) avec $S = \llbracket 0; n-1 \rrbracket$ et $A = \emptyset$;
- connaître le nombre de sommets d'un graphe (donc en pratique l'ensemble des sommets, puisqu'il s'agit des entiers de 0 à $n-1$) ;
- étant donné $(\llbracket 0; n-1 \rrbracket, A)$ un graphe et $u, v \in \llbracket 0; n-1 \rrbracket$, déterminer si l'arc (u, v) est présent, l'ajouter, ou le supprimer.
- étant donné (S, A) un graphe et $u \in S$, obtenir la liste des successeurs de u ;
- obtenir la liste des arcs d'un graphe.

Idéalement, toutes ces opérations sauf la création d'un nouveau graphe (qui nécessite de créer un tableau de taille n) et le calcul de la liste des arcs devraient se faire en temps constant ; malheureusement, ce n'est pas possible de tout avoir.

Il existe deux façons standard d'implémenter un graphe de façon à avoir des complexités raisonnables :

- *Matrice d'adjacence* : on représente le graphe par une matrice de booléens de taille $n \times n$, avec le coefficient de coordonnées (u, v) qui est `true` si et seulement si $u \rightarrow v$.
- *Listes d'adjacence* : on représente le graphe par un tableau de n listes d'entiers. La case d'indice u contient la liste des successeurs de u dans l'ordre croissant².

On donne également dans le fichier `TP21.ml` des test sur ces fonctions, et une fonction `est_oriente` qui prend en argument un graphe orienté et renvoie `true` si et seulement si ce n'est pas le graphe orienté correspondant à un graphe non orienté par l'injection canonique.

1. Dans un fichier `graphes.ml`, écrire une implémentation de la structure de graphe orientée en utilisant les matrices d'adjacence. Les complexités attendues sont les suivantes, en fonction de n le nombre de sommets :

- `creer` : $\Theta(n^2)$
- `taille` : $\Theta(1)$
- `ajoute_arc`, `supprime_arc` et `est_arc` : $\Theta(1)$
- `successeurs` : $\Theta(n)$
- `arcs` : $\Theta(n^2)$

Tester votre implémentation avec `TP21.ml`.

2. Changer le nom du fichier précédent, et dans un nouveau fichier `graphes.ml` écrire une implémentation utilisant les listes d'adjacence.

Les complexités attendues sont les suivantes, en fonction de n le nombre de sommets et m le nombre d'arcs :

- `creer` : $\Theta(n)$
- `taille` : $\Theta(1)$
- `ajoute_arc` et `supprime_arc` et `est_arc` : $\Theta(d_+(u))$ où u est le premier sommet de l'arc.
- `successeurs` : $\Theta(1)$
- `arcs` : $\Theta(n + m)$

Tester votre implémentation avec `TP21.ml`.

1. Qui peut donc également représenter un graphe non orienté : il suffit de garantir qu'il n'existe pas d'arc d'un sommet vers lui-même, et que si $u \rightarrow v$ alors $v \rightarrow u$ pour tous sommets u et v .

2. L'ordre peut être arbitraire, mais maintenir un ordre croissant ne coûte pas plus cher en complexité.

3. Avec chacune des deux implémentations, combien de mots mémoire sont nécessaires pour représenter les graphes suivants ?
- Un cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow (n-1) \rightarrow 1$.
 - C_n , un cycle non orienté à n sommets.
 - K_n graphe non orienté complet (c'est à dire avec toutes les arêtes possibles) à n sommets.
 - $K_{n,n}$, graphe à $2n$ sommets dont les n^2 arêtes sont les $\{i; n+j\}$ pour $i, j \in \llbracket 0; n-1 \rrbracket$.
4. On suppose qu'on change l'implémentation (c'est plus facile à faire en C) de façon à n'utiliser qu'un bit pour un booléen, et 32 bits pour un entier. Même question que ci-dessus.

Exercice 2 :**Parcours en profondeur**

Le parcours en profondeur d'un graphe depuis un $s \in S$ consiste à marquer la s comme exploré puis :

- terminer si tous les successeurs de la source sont explorés ;
- sinon, faire récursivement le parcours en profondeur depuis chacun des successeurs non encore explorés.

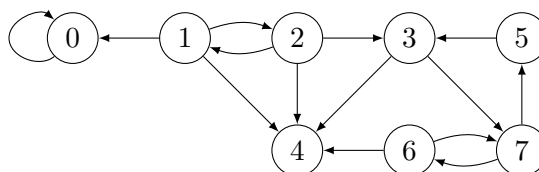
On donne ici un exemple de fonction réalisant tel parcours, qui renvoie un tableau de booléen indiquant si chaque sommet a été visité.

```

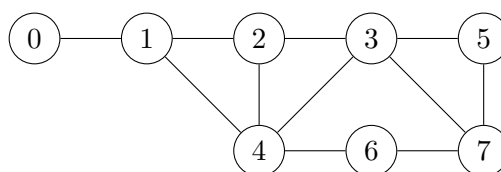
1 let dfs (g : graphe) (source : int) : bool array =
2   let visite = Array.make (taille g) false in
3   let rec dfs_aux s =
4     if not visite.(s) then
5       begin
6         visite.(s) <- true;
7         List.iter dfs_aux (successeurs g s)
8       end
9   in
10  dfs_aux source;
11  visite

```

1. On suppose que la **successeurs** renvoie la liste des successeurs dans l'ordre croissant :
- Écrire l'arbre d'appel de **dfs_aux(7)** sur le graphe ci-dessous, et dessiner pour chaque sommet visité autre que 7 l'arc par lequel ce sommet est visité³. Que remarque-t-on ?
 - Faire de même avec **dfs_aux(2)**.



- Écrire l'arbre d'appel de **dfs_aux(7)** sur le graphe ci-dessous.



2. Pourquoi ne peut-on pas échanger les lignes **visite.(v) <- true** et **List.iter dfs_aux (successeurs g v)** dans la fonction **dfs** ?
- En d'autres termes, pourquoi le parcours en profondeur doit-il être préfixe plutôt que suffixe ?

3. Ceci correspond à un *arbre couvrant* du sous-graphe induit par les sommets accessibles depuis 7.